

Hash Maps using `std::unordered_map`

Flynn Yuying Guo

Ref1:

Chapter 12 of "Building Low Latency Applications with C++" by Sourav Ghosh

Ref2:

StudyPlan.dev — Hash Maps using `std::unordered_map`

Agenda

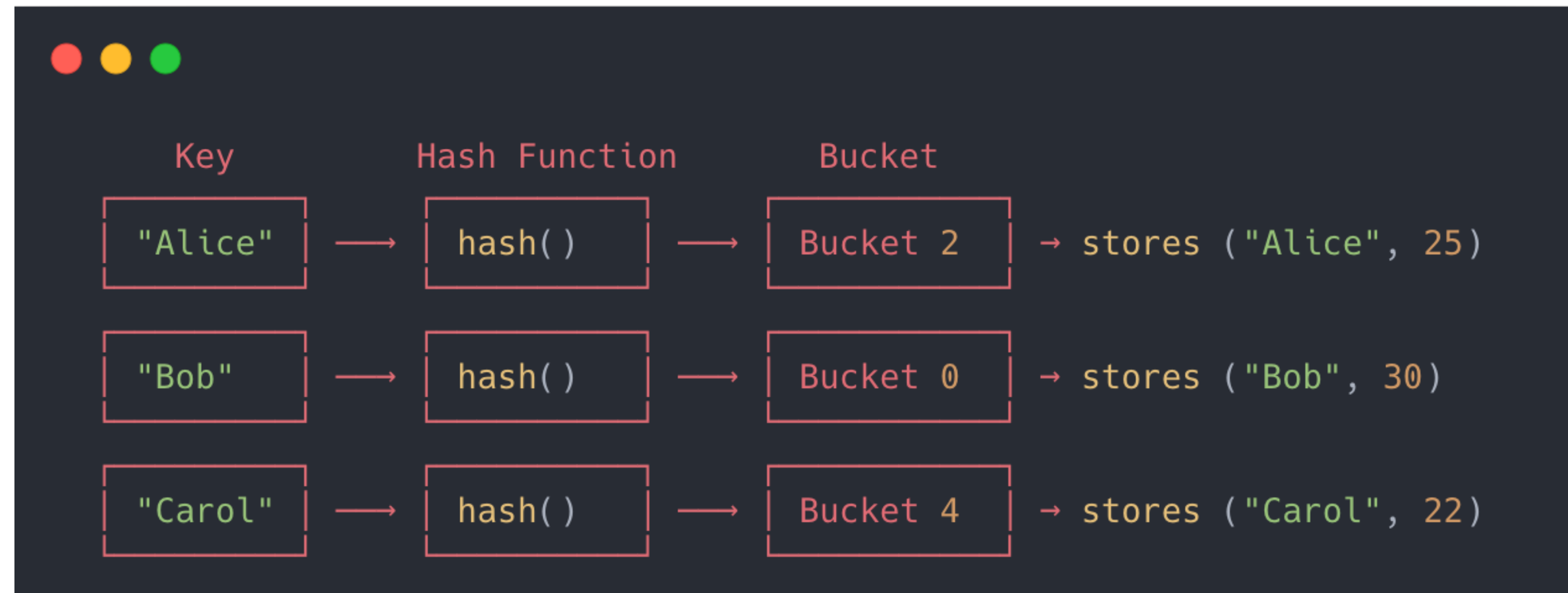
1. Concepts
2. Fundamentals
3. Advanced
4. Real World

1. Concepts

What is a Hash Map? · Buckets · Load Factor · Rehashing

What is a Hash Map?

A hash map stores **key-value pairs**, and lets you look up any value by its key in near-instant time.



How it works:

1. You provide a **Key** (e.g. "Alice")
2. A **hash function** converts the key into a number (e.g. 2)
3. That number is the **bucket index**: the data lives in that bucket
4. Next time you look up "Alice", it goes straight to Bucket 2, **no scanning required**

What is a Hash Map?

What is `std::pair`?

Every element inside an `unordered_map` is a `std::pair`: a simple struct that bundles two values together. You'll see `.first` and `.second` everywhere when working with maps.



```
std::pair<std::string, int> p{"Alice", 25};  
// p.first == "Alice"    (the Key)  
// p.second == 25        (the Value)
```

Time Complexity

Operation	Average	Worst Case
Lookup	$O(1)$	$O(n)$
Insert	$O(1)$	$O(n)$
Delete	$O(1)$	$O(n)$

$O(1)$ means the speed is roughly constant regardless of how many elements you have, 100 or 10 million.

The worst case $O(n)$ only occurs when all keys hash to the same bucket (extremely rare with a decent hash function).

Buckets

Internally, `unordered_map` maintains an array of **buckets**. Each bucket can hold one or more elements (via chaining, typically a linked list):

```
● ● ●

Bucket 0: → ("Bob", 30)
Bucket 1: → (empty)
Bucket 2: → ("Alice", 25) → ("Eve", 28) ← collision: two elements in one bucket
Bucket 3: → (empty)
Bucket 4: → ("Carol", 22)
...
Bucket 7: → (empty)
```

`bucket_count()` — How Many Buckets?

```
● ● ●

std::unordered_map<std::string, std::string> m{
    {"Bob", "bob@example.com"},
    {"Anna", "anna@example.com"},
    {"Dave", "dave@example.com"}
};

std::cout << "Buckets: " << m.bucket_count();
// Output: Buckets: 8 (exact number is implementation-defined)
```


Load Factor

$\text{load_factor} = \text{number_of_elements} / \text{bucket_count}$

- Higher load factor → buckets are crowded → more collisions → slower lookups
- Lower load factor → buckets are sparse → wastes memory → faster lookups

load_factor() — Current Load

```
std::cout << "Load Factor: " << m.load_factor();  
// 3 elements / 8 buckets = 0.375
```

max_load_factor() — The Rehash Threshold

When `load_factor()` exceeds `max_load_factor()`, the container automatically rehashes. The default threshold is **1.0**.

```
// Check current threshold  
std::cout << "Max Load Factor: " <<  
m.max_load_factor(); // 1  
  
// Set a tighter threshold  
m.max_load_factor(0.25);  
  
// Now inserting a new element may trigger a  
rehash  
m["Steve"] = "steve@example.com";  
std::cout << "Load Factor: " << m.load_factor();  
// Might output: 0.0625 (4 elements / 64 buckets)
```


Rehashing

When the load factor exceeds the threshold, the container:

- Allocates a **larger bucket array**
- **Re-hashes every element** into the new array
- Deallocates the old array

This is an **O(n)** operation, very expensive! In performance-critical code (e.g. HFT), you want to avoid this happening at runtime.

rehash(n) — Manually Set Minimum Bucket Count

set the bucket count directly to $\geq n$

```
std::cout << "Buckets: " << m.bucket_count();  
// 8  
  
m.rehash(500);  
  
std::cout << "Buckets: " << m.bucket_count();  
// 512 (rounds to nearest power of 2)
```

reserve(n) — ★ Pre-allocate for n Elements (Most Important!)

set bucket count so that n elements fit without triggering a rehash
(accounts for max_load_factor)

```
m.max_load_factor(0.25);  
m.reserve(500); // allocate enough buckets for 500  
elements  
  
std::cout << "Buckets: " << m.bucket_count(); // 2048  
 (= 500 / 0.25)
```


2.

Fundamentals

Create • Access • Insert • Delete • Edit • Iterate

Creating and Initializing

Method 1: Empty Map

Reading the template parameters:
`unordered_map<string, string>` — first type is the Key, second is the Value. You can use any types, e.g. `unordered_map<int, double>`.

Method 2: Initializer List (Most Common)

Each `{"Alice", "alice@example.com"}` is actually a `std::pair<string, string>` being constructed inline.

```
#include <unordered_map>
#include <string>

int main() {
    // Key type: string, Value type: string
    std::unordered_map<std::string, std::string> contacts;
    // contacts is empty, size = 0
}
```

```
std::unordered_map<std::string, std::string> contacts{
    {"Alice", "alice@example.com"},
    {"Bob", "bob@example.com"},
    {"Carol", "carol@example.com"}
};
```

Creating and Initializing

Method 3: From Existing Pairs

Method 4: CTAD — Let the Compiler Deduce Types (C++17)

CTAD = Class Template Argument Deduction.

```
using std::string;
using Person = std::pair<string, string>;

Person alice{"Alice", "alice@example.com"};
Person bob{"Bob", "bob@example.com"};
Person carol{"Carol", "carol@example.com"};

std::unordered_map<string, string>
contacts{alice, bob, carol};
```

```
using namespace std::string_literals;

std::pair a{"Alice"s, "alice@example.com"s};
std::pair b{"Bob"s, "bob@example.com"s};

// No need to write <string, string> – the
// compiler figures it out
std::unordered_map contacts{a, b};
```


Querying Size: `size()` and `empty()`

`size()` — How Many Elements?

```
std::unordered_map<std::string, int> ages{
    {"Alice", 25}, {"Bob", 30}
};

std::cout << ages.size(); // Output: 2
```

`empty()` — Is It Empty?

```
std::unordered_map<int, int> m;

if (m.empty()) {
    std::cout << "Empty!"; // This prints
}
```

Accessing Elements: 5 Methods

Method 1. operator[] — Most Common, But Has a Side Effect

```
std::unordered_map<std::string, std::string> m{
    {"Alice", "alice@example.com"},
    {"Bob", "bob@example.com"}
};

// ✅ Reading an existing key
std::cout << m["Alice"]; // Output: alice@example.com

// ⚠️ DANGER: accessing a non-existent key silently inserts a default value!
std::cout << m["Steve"]; // Output: (empty string)
// m now has 3 elements – {"Steve", ""} was auto-inserted!
```

Remember this trap:

[] on a missing key doesn't throw, it silently inserts a default value. This is a common source of bugs. Use at(), find(), or contains() if you don't want this behavior.

Accessing Elements: 5 Methods

Method 2. at() — Safe Version, Throws on Missing Key

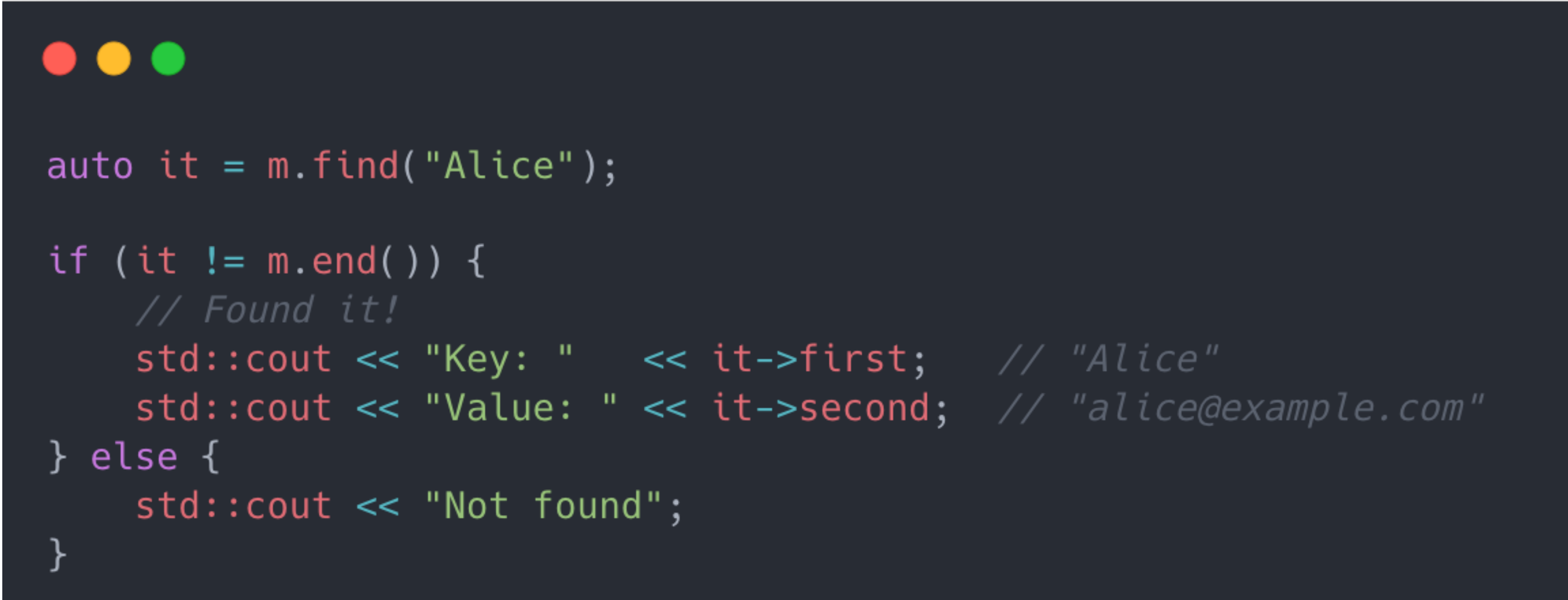


```
std::cout << m.at("Alice"); // ✅ Output: alice@example.com

try {
    std::cout << m.at("Steve"); // ✨ throws std::out_of_range!
} catch (std::out_of_range& e) {
    std::cout << "Steve not found: " << e.what();
}
```


Accessing Elements: 5 Methods

Method 3. find() — Returns an Iterator, Most Flexible



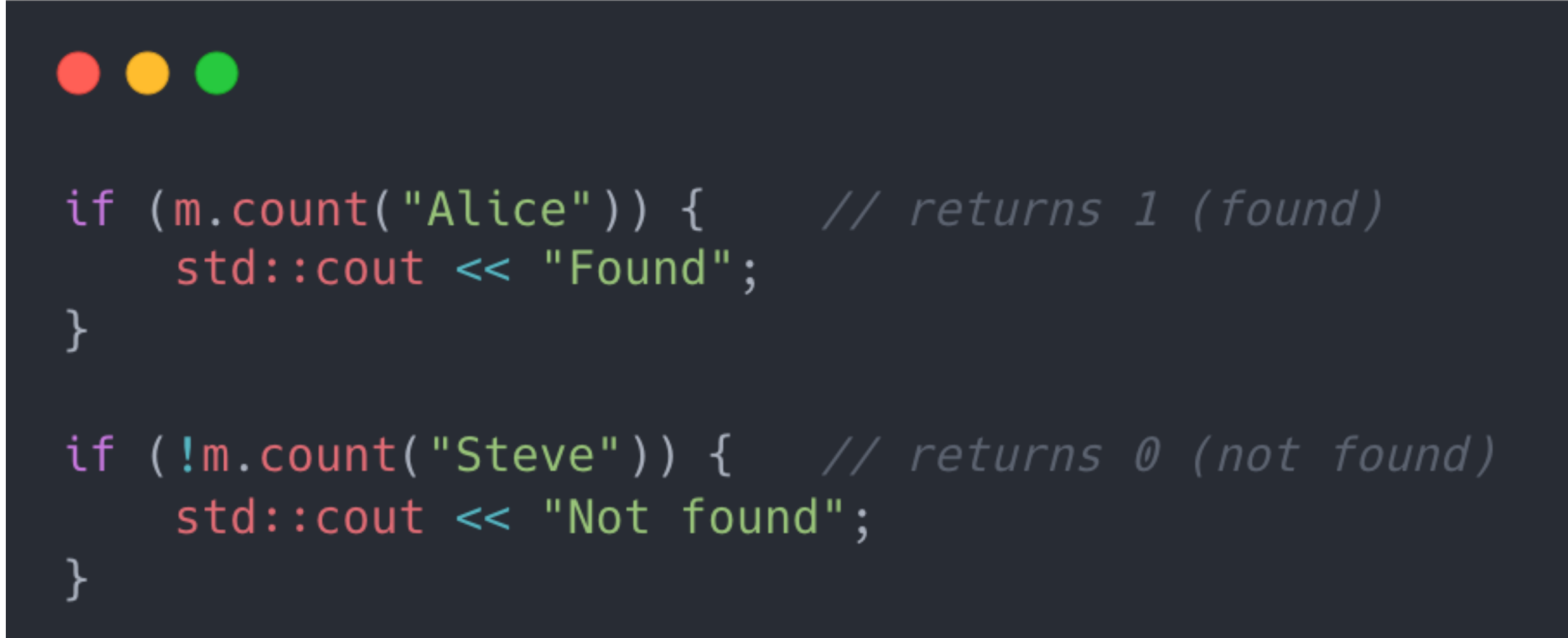
```
auto it = m.find("Alice");

if (it != m.end()) {
    // Found it!
    std::cout << "Key: "    << it->first;    // "Alice"
    std::cout << "Value: " << it->second;   // "alice@example.com"
} else {
    std::cout << "Not found";
}
```

- If the key doesn't exist, find() returns m.end() (a special past-the-end iterator)
- find() is the safest and most flexible option: check-then-use without side effects.

Accessing Elements: 5 Methods

Method 4. `count()` — Pre-C++20 Alternative to `contains()`



```
if (m.count("Alice")) {    // returns 1 (found)
    std::cout << "Found";
}

if (!m.count("Steve")) {  // returns 0 (not found)
    std::cout << "Not found";
}
```

Since `unordered_map` doesn't allow duplicate keys, `count()` always returns 0 or 1. 0 converts to false, 1 to true.

Accessing Elements: 5 Methods

Method 5. `contains()` — Simplest, Just Ask "Is It There?" (C++20)



```
if (m.contains("Alice")) {  
    std::cout << "Found Alice";  
}  
  
if (!m.contains("Steve")) {  
    std::cout << "Steve not found";  
}
```

Added in C++20. If your compiler supports it, prefer this for existence checks.

Accessing Elements: 5 Methods

Comparison Table

Method	Syntax	If Key Missing	Best For
[]	m[key]	⚠ Silently inserts default	You're sure the key exists, or you want to insert
at()	m.at(key)	💣 Throws exception	Key should exist; missing = bug
find()	m.find(key)	Returns m.end()	Need both existence check and value
count()	m.count(key)	Returns 0	Just checking existence (pre-C++20)
contains()	m.contains(key)	Returns false	Just checking existence (C++20)

Inserting Elements: 5 Methods

Method 1. operator[] +=

```
m["Steve"] = "steve@example.com";  
// Key doesn't exist → inserts new entry  
// Key already exists → overwrites the old value  
std::unordered_map<std::string, std::string> m{  
    {"Alice", "alice@example.com"},  
    {"Bob",    "bob@example.com"},  
    {"Dave",   "dave@example.com"}  
};  
// Duplicate Key Rules  
// unordered_map does not allow duplicate keys. Using [] on an existing key overwrites it:  
std::cout << "Size: " << m.size();           // 3  
m["Bob"] = "new-bob@example.com";             // overwrites Bob's value  
std::cout << "\nSize: " << m.size();           // still 3!  
std::cout << "\nBob: " << m["Bob"];            // new-bob@example.com  
// However, different keys can share the same value:  
m["Robert"] = "bob@example.com"; // same email as Bob, different key  
std::cout << m.size();                 // now 4  
  
if (m["Bob"] == m["Robert"]) {  
    std::cout << "Bob and Robert share the same value";  
}
```


Inserting Elements: 5 Methods

Method 2. insert() — Skips if Key Already Exists



```
// Using an inline pair
m.insert({"Steve", "steve@example.com"});

// Using a named pair
std::pair<std::string, std::string> steve{"Steve", "steve@example.com"};
m.insert(steve);

// ⚠ If the key already exists, insert does NOTHING (no overwrite)
m.insert({"Alice", "new@example.com"});
// Alice's value is still "alice@example.com"
```


Inserting Elements: 5 Methods

Method 3. insert_or_assign() — Overwrites and Reports What Happened



```
auto [it, wasInserted] = m.insert_or_assign("Bob", "bob-new@example.com");

if (wasInserted) {
    std::cout << "Inserted new entry: " << it->first;
} else {
    std::cout << it->first << " already existed, value updated to " << it->second;
    // Output: Bob already existed, value updated to bob-new@example.com
}
```

Returns `pair<iterator, bool>`: the iterator points to the element, the bool is true if newly inserted, false if updated. Combines the overwrite behavior of `[]` with the feedback of `insert`.

Inserting Elements: 5 Methods

Method 4. `emplace()` — More Efficient `insert()`



```
m.emplace( "Steve", "steve@example.com" );
```

`emplace` constructs the pair **in-place** inside the container, avoiding the overhead of creating the pair externally then copying/moving it in. Same behavior as `insert` (skips if key exists), but more efficient.

Inserting Elements: 5 Methods

Method 5. try_emplace() — Emplace with Feedback



```
// "Steve" doesn't exist → inserted successfully
m.try_emplace("Steve", "steve@example.com");

// "Bob" already exists → skipped, Bob's value unchanged
m.try_emplace("Bob", "rob@example.com");

std::cout << "Steve: " << m["Steve"]; // steve@example.com
std::cout << "Bob: " << m["Bob"];    // still the original value

//try_emplace also returns pair<iterator, bool>:
auto [it, success] = m.try_emplace("Bob", "rob@example.com");

if (!success) {
    std::cout << "Key " << it->first << " already exists";
    std::cout << ", value is: " << it->second;
}
```


Inserting Elements: 5 Methods

Comparison Table

Method	If Key Exists	Returns Info?
<code>m[key] = val</code>	 Overwrites	No
<code>insert({key, val})</code>	 Skips	<code>pair(iterator, bool)</code>
<code>insert_or_assign(key, val)</code>	 Overwrites	<code>pair(iterator, bool)</code>
<code>emplace(key, val)</code>	 Skips	<code>pair(iterator, bool)</code>
<code>try_emplace(key, val)</code>	 Skips	<code>pair(iterator, bool)</code>

Removing Elements: `erase()` and `clear()`

`erase(key)` — Remove a Specific Key

`erase()` returns the number of elements

removed: 1 if found and removed, 0 if not found.

`clear()` — Remove All Elements



```
std::cout << "Size: " << m.size(); // 3
m.erase("Bob");
std::cout << "Size: " << m.size(); // 2

// If the key doesn't exist, erase does nothing (no error)
m.erase("NonExistent"); // safe, returns 0

//-----
//-----
//-----

std::cout << "Size: " << m.size(); // 3
m.clear();
std::cout << "Size: " << m.size(); // 0
```

Editing Values



```
std::unordered_map<std::string, int> scores{  
    {"a", 1}, {"b", 2}, {"c", 3}  
};
```

```
// Direct assignment  
scores["a"] = 100;
```

```
// Using at()  
scores.at("b") = 200;
```

```
// Increment, compound assignment  
++scores["a"];      // 100 → 101  
scores["b"] += 10;   // 200 → 210
```

```
// Pass by reference to another function  
void doubleIt(int& x) { x *= 2; }  
doubleIt(scores["c"]); // 3 → 6
```

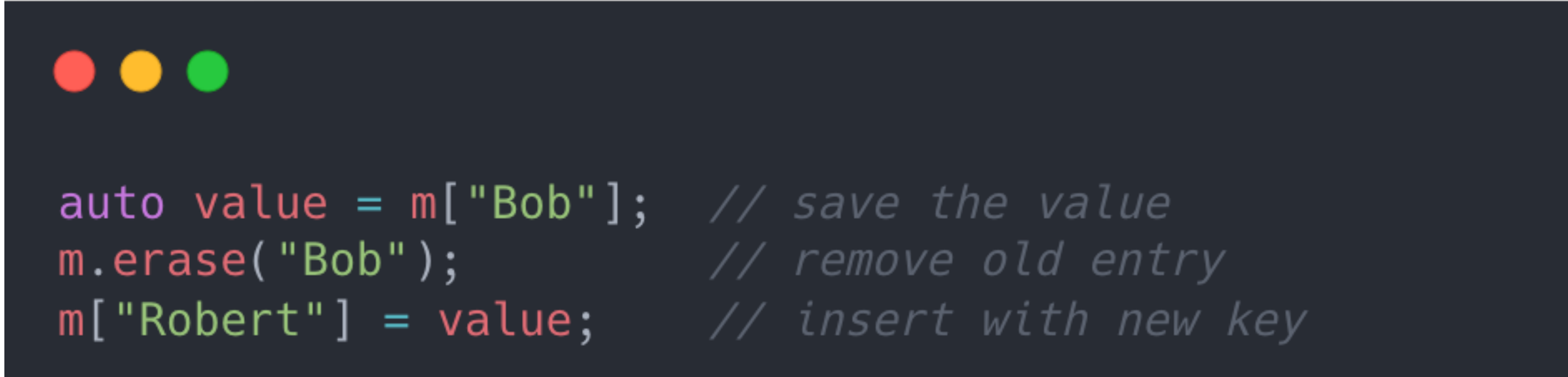
```
// Result:  
// a=101, b=210, c=6
```


Editing Keys: `extract()` (C++17)

Why You Can't Edit Keys Directly

The key determines which bucket an element lives in (via the hash function). If you modify a key in place, the element is now in the wrong bucket and can never be found again.

The Naive Way: Erase and Re-insert



```
auto value = m["Bob"]; // save the value
m.erase("Bob");       // remove old entry
m["Robert"] = value;   // insert with new key
```

This works but has a downside: the Value is destroyed and recreated. If the Value type is expensive to copy or has side effects in its destructor, this is wasteful.

Editing Keys: `extract()` (C++17)

The Better Way: `extract()` (C++17)

`extract()` removes the node from the container without destroying the objects inside. You can modify the key, then re-insert

Why `extract()` is better:

- The Value object is never destroyed or copied, it stays in place
- The container automatically re-hashes the modified key on insertion
- Much more efficient for large or complex Value types

```
#include <iostream>
#include <unordered_map>
#include <string>

int main() {
    std::unordered_map<std::string, std::string> m{
        {"Bob", "bob@example.com"},
        {"Anna", "anna@example.com"},
        {"Dave", "dave@example.com"}
    };

    // Step 1: Extract the node (removes it from the map)
    auto node = m.extract("Bob");
    // m now only has Anna and Dave

    // Inspect the extracted node
    std::cout << "Key: " << node.key()
              << ", Value: " << node.mapped() << "\n";
    // Output: Key: Bob, Value: bob@example.com

    // Step 2: Modify the key
    node.key() = "Robert";

    // Step 3: Re-insert (must use std::move)
    m.insert(std::move(node));

    // m now has Anna, Dave, Robert
    std::cout << "Robert: " << m["Robert"];
    // Output: Robert: bob@example.com
}
```

Iterating Over a Map

Method 1: By Value (Copies Each Element)

⚠ Each element is copied into pair. This is fine for small types like int, but wasteful for string or large objects.

Method 2: By Const Reference (Recommended)

& → pass by reference, avoids copying

Method 3: Structured Binding (C++17, Best)

[key, value] automatically unpacks the pair's first and second into named variables. This is the cleanest and most modern approach.

```
// Method 1
for (auto pair : m) {
    std::cout << "Key: " << pair.first
               << ", Value: " << pair.second << "\n";
}

// Method 2
for (const auto& pair : m) {
    std::cout << "Key: " << pair.first
               << ", Value: " << pair.second << "\n";
}

// Method 3
for (const auto& [key, value] : m) {
    std::cout << "Key: " << key
               << ", Value: " << value << "\n";
}
```


Iterating Over a Map

⚠ Iteration Order is Unpredictable!

This is what **unordered** means. If you need sorted iteration by key, use `std::map` (red-black tree, always sorted).

```
std::unordered_map<std::string, int> m{
    {"Alice", 1}, {"Bob", 2}, {"Carol", 3}
};

for (const auto& [k, v] : m) {
    std::cout << k << " ";
}

// Possible output: Bob Carol Alice
// Or: Alice Carol Bob
// Or any other order – it may differ between runs!
```

3. Advanced

Custom Types · Custom Hash · Custom Comparison

Using Custom Types as Keys

enum class as Key (Most Common Scenario)

When your keys are a fixed set of options, enum class is ideal.

Why does enum class work as a Key out of the box?

Because it's backed by an integer type, and the standard library already provides `std::hash` for integers.

Using Other Custom Types as Keys

For a custom struct as Key, you need to provide two things:

- A hash function — tells the container how to convert the Key into a number
- An equality comparison — tells the container how to determine if two Keys are the same

```
#include <iostream>
#include <unordered_map>
#include <string>

enum class Slot { Weapon, Armor, Helmet };
//                      0         1         2

struct Item {
    std::string Name;
};

int main() {
    std::unordered_map<Slot, Item> equipment{
        {Slot::Weapon, Item{"Iron Sword"}},
        {Slot::Armor, Item{"Leather Armor"}},
        {Slot::Helmet, Item{"Wooden Bucket"}}
    };

    std::cout << "Helmet: "
              << equipment[Slot::Helmet].Name;
    // Output: Helmet: Wooden Bucket
}
```


Custom Hash Functions

If you can't (or don't want to) specialize `std::hash`, you can write a separate hash functor and pass it as the **third template argument**.

What is a functor?

A struct/class that overloads `operator()`. Its instances can be "called" like a function: `UserHash h; h(someUser);`

```
#include <unordered_map>
#include <iostream>
#include <string>

struct User {
    std::string Name;

    bool operator==(const User& other) const {
        return Name == other.Name;
    }
};

// A functor (function object) for hashing
struct UserHash {
    size_t operator()(const User& u) const {
        return std::hash<std::string>{}(u.Name);
    }
};

int main() {
    // Third template argument: the hash function type
    std::unordered_map<User, std::string, UserHash> m;
    m[User{"Bob"}] = "bob@example.com";

    std::cout << "Bob: " << m[User{"Bob"}]; // bob@example.com
}
```

Custom Comparison Functions

When two different keys hash to the same value (a collision), the container needs a way to tell them apart. By default it uses `operator==`, but you can customize this via the **fourth template argument**.

⚠ Critical rule: If the comparator says `A == B` (true), then the hash function **must** return the same value for A and B. Violating this rule causes undefined behavior — the map will silently malfunction.

```
#include <unordered_map>
#include <iostream>
#include <string>

struct User {
    std::string Name;
    // Note: no operator== defined here
};

struct UserHash {
    size_t operator()(const User& u) const {
        return std::hash<std::string>{}(u.Name);
    }
};

// Custom comparison functor
struct UserEqual {
    bool operator()(const User& a, const User& b) const {
        return a.Name == b.Name;
    }
};

int main() {
    // Four template arguments: Key, Value, Hash, Compare
    std::unordered_map<User, std::string, UserHash, UserEqual> m;
    m[User{"Bob"}] = "bob@example.com";

    std::cout << "Bob: " << m[User{"Bob"}]; // bob@example.com
}
```

4. Real World

HFT Case Study

HFT Case Study: array vs unordered_map

The Problem: An Exchange Order Book

In a high-frequency trading (HFT) matching engine, the order book needs to:

- Look up orders by ClientId + OrderId
- Find all orders at a given Price level

This requires fast key-value lookups, a textbook use case for a hash map, right?

HFT Case Study: array vs unordered_map

Approach A: std::array as a "Fake Hash Map"

```
  
// Precondition: ClientId ranges from 0 to ME_MAX_NUM_CLIENTS (e.g. 256)  
// Precondition: OrderId ranges from 0 to ME_MAX_ORDER_IDS (e.g. 65536)  
  
using OrderHashMap = std::array<MEOrder*, ME_MAX_ORDER_IDS>;  
using ClientOrderHashMap = std::array<OrderHashMap, ME_MAX_NUM_CLIENTS>;  
  
ClientOrderHashMap cid_oid_to_order_;  
  
// Lookup: direct index, O(1), zero overhead  
MEOrder* order = cid_oid_to_order_[client_id][order_id];
```

Approach B: std::unordered_map

```
  
  
std::unordered_map<ClientId,  
    std::unordered_map<OrderId, MEOrder*>> cid_oid_to_order_;  
  
// Lookup: hash-based, O(1) average  
MEOrder* order = cid_oid_to_order_[client_id][order_id];
```


HFT Case Study: array vs unordered_map

	array	unordered_map
Memory layout		
	Contiguous block	Nodes scattered on heap
Cache friendly	Excellent 	Poor 
Access method	Direct index (1 step)	hash → bucket → chase pointer (multi-step)
Dynamic alloc	None	Every node is heap-allocated

Benchmark Results:

ARRAY HASHMAP 142,650 CLOCK CYCLES PER OPERATION

UNORDERED-MAP HASHMAP 152,457 CLOCK CYCLES PER OPERATION

unordered_map is ~6–7% slower.

HFT Case Study: array vs unordered_map

The key concept: Cache Locality

CPUs don't read memory one byte at a time — they load entire cache lines (typically 64 bytes). With `std::array`, neighboring elements are loaded into cache automatically. With `unordered_map`, each node may live at a completely different memory address, causing cache misses that require fetching from main memory.

Memory Level	Access Latency
CPU Registers	~1 cycle
L1 Cache	~4 cycles
L2 Cache	~10 cycles
L3 Cache	~40 cycles
Main Memory (DRAM)	~200 cycles ← <code>unordered_map</code> often ends up here

// The gap from L1 to main memory is ~50x. This is why "O(1) ≠ fast" — Big-O notation ignores constant factors and cache effects.

The Tradeoff: Array's Downsides

- **Wastes memory** if the key range is large but sparsely used
- **Cannot grow dynamically** — the size is fixed at compile time
- Only works when the **key range is known and small** (e.g. `ClientId` 0–255)

HFT Case Study: array vs unordered_map

Other Hash Map Alternatives

The book mentions several alternatives that sit between array and unordered_map in terms of cache friendliness: `absl::flat_hash_map`, `robin_hood::unordered_map`, `tsl::hopscotch_map`, and others. These use **open addressing** (storing elements inline in the bucket array rather than in separate heap nodes), which gives much better cache behavior than `std::unordered_map`.

Scenario	Best Choice
Key is integer, range known & small	<code>std::array</code>
Key is string or range unknown	<code>std::unordered_map</code>
Extreme performance (HFT hot path)	<code>std::array</code> or <code>absl::flat_hash_map</code>
General purpose, flexibility first	<code>std::unordered_map</code>
Need sorted iteration by key	<code>std::map</code>

Summary

Quick Reference • Key Takeaways

Quick Reference

```

● ● ●

#include <unordered_map>
#include <string>

std::unordered_map<std::string, int> m;

// ===== INSERT =====
m["Alice"] = 25; // insert or overwrite
m.insert({"Bob", 30}); // insert (skip if exists)
m.emplace("Carol", 22); // in-place insert (skip if exists)
m.try_emplace("Dave", 28); // insert with feedback (skip if exists)
m.insert_or_assign("Eve", 26); // insert or overwrite, with feedback

// ===== ACCESS =====
int age = m["Alice"]; // ⚠ inserts default if missing
int age = m.at("Alice"); // throws if missing
auto it = m.find("Alice"); // returns iterator, m.end() if missing
bool has = m.contains("Alice"); // C++20
bool has = m.count("Alice"); // pre-C++20

// ===== MODIFY =====
m["Alice"] = 100; // modify value
++m["Alice"]; // increment
auto node = m.extract("Alice"); // modify key (C++17)
node.key() = "Alicia";
m.insert(std::move(node));

// ===== DELETE =====
m.erase("Alice"); // remove one key
m.clear(); // remove all

// ===== ITERATE =====
for (const auto& [key, value] : m) {
    std::cout << key << ": " << value << "\n";
}

// ===== SIZE =====
m.size(); // element count
m.empty(); // is it empty?

// ===== PERFORMANCE =====
m.reserve(1000); // ★ pre-allocate, avoid rehash
m.bucket_count(); // number of buckets
m.load_factor(); // current load factor
m.max_load_factor(0.5); // set rehash threshold
m.rehash(2000); // manually set min bucket count
```


Custom Key Types Quick Reference



```
// Approach 1: Specialize std::hash + define operator==
struct MyKey {
    int id;
    bool operator==(const MyKey& o) const { return id == o.id; }
};
namespace std {
    template<> struct hash<MyKey> {
        size_t operator()(const MyKey& k) const {
            return std::hash<int>{}(k.id);
        }
    };
}
std::unordered_map<MyKey, std::string> m1;

// Approach 2: Custom Hash and Compare functors
struct MyHash {
    size_t operator()(const MyKey& k) const { return std::hash<int>{}(k.id); }
};
struct MyEqual {
    bool operator()(const MyKey& a, const MyKey& b) const { return a.id == b.id; }
};
std::unordered_map<MyKey, std::string, MyHash, MyEqual> m2;
```

Key Takeaways

- **operator[] silently inserts** — use `find()` or `contains()` when you're not sure the key exists
- **insert() skips, [] overwrites** — pick the right one for your intent
- **Use extract() to modify keys** — never modify a key in place (C++17)
- **Custom key types need a hash function + equality** — and they must be consistent
- **Call reserve() when you know the size** — avoids expensive mid-operation rehashing
- **Iteration order is unpredictable** — use `std::map` if you need sorted keys
- **$O(1)$ $\neq$ fastest in practice** — cache locality may make `std::array` or `flat_hash_map` faster

Questions & Discussions

Thanks for listening!